유경동

Unity·C# 게임 클라이언트 개발자 · 씨플레이 CatJump · 2025.07~재직 중(경력 1년+)

씨플레이에서 CatJump의 보상·이벤트·10인 PvP 클라이언트를 개발하고 있습니다. 이벤트마다 하드코딩하던 대사를 공용 시스템으로 설계·구현했고, PvP의 호스트 변경 복구와 보상 지급·표시 분리를 구현했습니다.

- 이메일 yugd1211@gmail.com
- GitHub github.com/yugd1211
- 영상 youtube.com/@k_1q2w3e4r
- 서울·경기·부산 근무 가능, 수도권 이주 가능

핵심 강점

10인 PvP의 호스트 변경 복구

진행 단계를 룸 속성에 기록해 새 마스터가 이어 가게 하고, 이탈한 자리는 AI로 대체했습니다. (3쪽)

이벤트별 대사를 공용 시스템으로

이벤트마다 하드코딩하던 대사·연출을 공용 시스템으로 처음부터 설계·구현했습니다. (4쪽)

보상 지급 이후 저장·표시까지 연결

지급·저장을 확정한 결과만 표시 계층에 넘기고, 추첨 전에 획득 불가 후보를 뺐습니다. (5쪽)

사례 바로가기

쪽	프로젝트	사례	구분
3	CatJump	PUN2 10인 PvP의 호스트 변경 복구, 순번 카운트다운, 이탈 AI 대체	회사 · 본인 전 범위 구현
4	CatJump	이벤트별 하드코딩 대사·연출을 공용 시스템으로	회사 · 본인 설계·구현
5	CatJump	보상 지급·저장과 표시 분리, 스킨 상태 연결	회사 · 담당 구현
6	CatJump	회전 UI의 월드 OBB 판정, 진행 규칙과 표현 분리	회사 · 담당 구현
7	CatJump	광고 복귀 저장 누락 수정과 라이브 운영 업무	회사 · 라이브 대응
8	DARK SHIELD	분기 스테이지 생성 범위를 3단계로 바꾼 기록	팀 · 팀장
9	Jumac	랭킹 동점 키 중복 수정, 인벤토리 변경 반영 경로	팀 · 팀장
10	NT	곡 에디터 데이터를 노트 생성 경로에 연결(변환·로드)	팀 · 개발팀장
12	IRC 서버 외	kqueue 서버(팀 구조), 1인 학습 프로젝트 2개	부록

기술 스택과 사용처는 11쪽에 정리했습니다.

CatJump

약 2~3주 간격으로 업데이트되는 CatJump에서 PvP·이벤트·보상을 개발하고 출시 후 오류를 수정합니다.



CatJump 공개 홍보 그래픽 · 출처: Google Play. 서비스 소개용이며 본인 구현의 증거는 아닙니다.

항목	내용
기간	2025.07~재직 중
회사	씨플레이 개발팀 · 사원
서비스	약 70만 MAU 라이브 모바일 게임 · Google Play 500만+ 다운로드
내 역할	Unity·C# 클라이언트 개발 · 라이브 운영 대응
기술	Unity · C# · UGUI · Photon PUN2 · Firebase
링크	Google Play

맡은 범위

구분	내용
본인 설계·구현	10인 실시간 PvP 전 범위, 공용 대사 시스템
담당 구현	보상 상자의 지급·표시 분리, 스킨 상태 연결, 회전 판정 미니게임
운영 대응	이벤트 10종 이상 개발·수정, 레벨 에디터 업데이트
수정 대응	UI 변경, 튜토리얼·연출 스크립트, 출시 후 오류



공개 스토어 화면 2컷(기본 플레이·온라인 io 플레이) · 출처: Google Play. 서비스 소개용입니다.

비공개 코드 정책. CatJump의 소스 코드·내부 문서·데이터는 싣지 않았고 제공하지 않습니다. 이 문서의 CatJump 구조도는 담당 기능의 흐름을 일반 명사로 다시 그린 설명 자료입니다. 사례의 결과에는 확인된 변경을 적고, 확인 가능한 시험·측정 기록이 없는 부분은 "기록 범위"로 따로 표시했습니다.

PUN2 10인 PvP: 호스트가 바뀌어도 룸 속성 기준으로 경기 복구

전용 서버 없는 PUN2 구조에서 마스터 클라이언트가 바뀌어도 경기를 이어 가야 했습니다.

문제

10인 레이스는 클라이언트마다 이벤트를 받는 시점이 다릅니다. 진행 권한을 가진 마스터 클라이언트가 나가면 준비·카운트다운·결과 처리가 멈추거나 어긋날 수 있습니다.

원인

전용 서버 없이 마스터 클라이언트가 진행을 맡는 PUN2 구조이고, 모바일 네트워크 환경입니다.

판단

진행 단계를 룸 속성에 기록해 호스트가 바뀐 뒤 읽을 복구 기준을 남겼습니다. 아래 비교는 지금 정리한 것입니다.

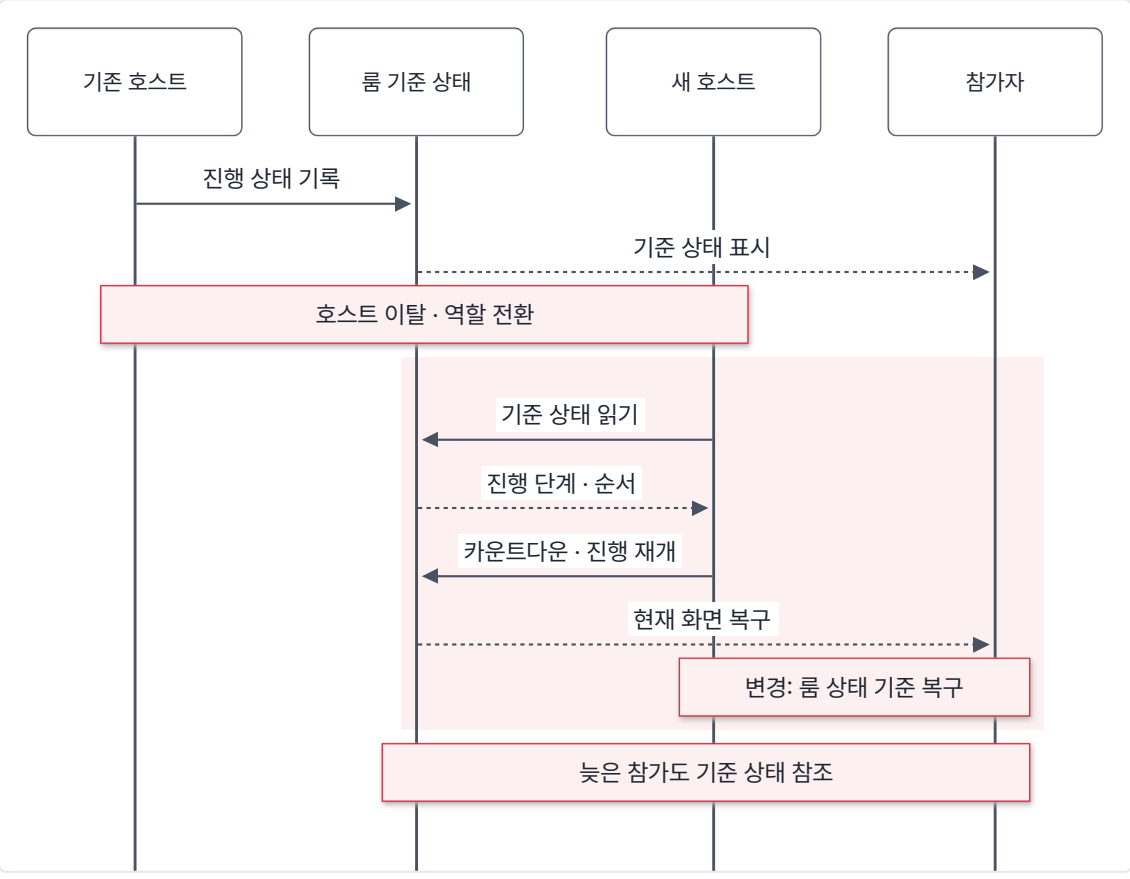
진행 단계 보관 위치	얻는 것	비용
클라이언트 메모리에만	구현이 단순함	새 마스터가 이전 단계의 기준을 읽을 수 없음
룸 속성에 기록 (선택)	새 마스터가 같은 값을 읽어 재개	갱신 지연·동시 갱신 충돌을 다뤄야 함

구현

매칭 탐색 → 인원 확정 → 준비·카운트다운 → 플레이 → 결과로 단계를 나눴습니다. 새 마스터는 룸 속성에서 진행 단계를 읽어 재개합니다. 카운트다운은 이와 별개로 이벤트에 순번을 붙여 순번이 지난 이벤트를 반영하지 않고, 이탈한 자리는 AI로 대체합니다.

결과

출시 후 카운트다운 불일치·룸 진입 크래시·로비 복귀 오류를 수정했습니다. (기록 범위: 지연·이탈 조합별 시험 기록과 네트워크 품질 측정치는 없음)



호스트 전환 복구(일반화): 새 호스트가 룸 상태를 읽어 진행과 참가자 화면을 맞춤(AI 대체는 제외).

이벤트마다 하드코딩하던 대사·연출을 공용 시스템으로 설계·구현

약 2~3주 주기로 이벤트가 바뀌는 운영에서, 이벤트마다 따로 만들던 대사·연출을 한 시스템으로 모았습니다.

문제

공용 시스템 이전에는 이벤트마다 대사와 연출을 코드에 직접 써서 만들었습니다. 이벤트가 바뀔 때마다 대사 UI 연결과 종료·예외 처리를 다시 작성해야 했습니다.

원인

새 이벤트가 붙을 때마다 같은 구현이 반복되는 구조였습니다. 공용화하려면 로컬라이징, 타이핑 중 리치 텍스트 태그, 뒤로가기 닫기를 함께 다뤄야 했고, 이미 하드코딩된 기존 이벤트와도 공존해야 했습니다.

판단

대사·연출을 이벤트 코드 밖으로 빼고 공용 시스템이 처리하게 했습니다. 표의 장단점 행은 지금 정리한 비교입니다.

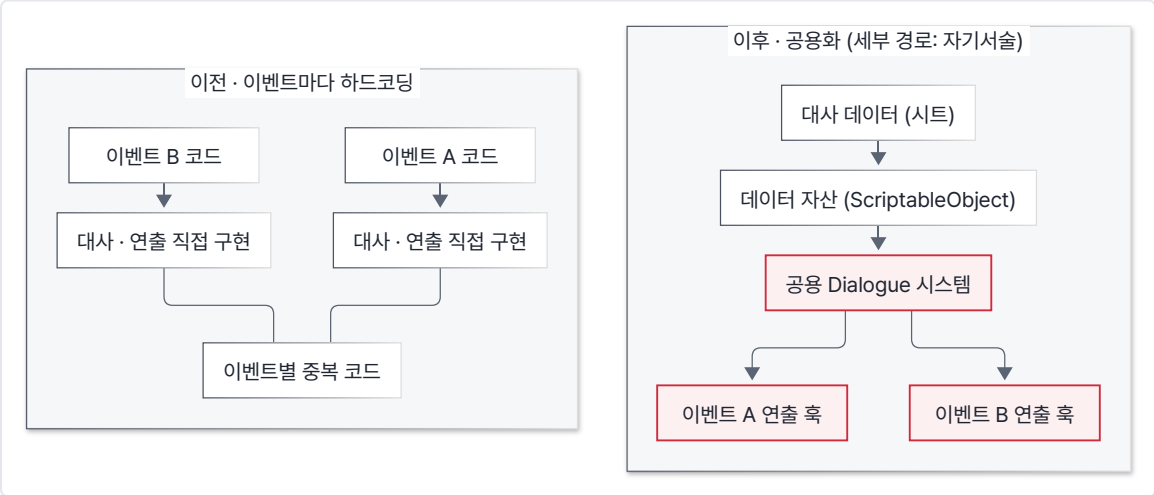
항목	이벤트별 하드코딩 (기존)	공용 대사 시스템 (변경)
대사·연출 정의	이벤트 코드 안	이벤트 코드 밖 데이터
종료 후 다음 동작	이벤트마다 직접 연결	완료 콜백으로 연결
장단점(사후 정리)	이벤트 하나만 보면 빠르고 연출이 자유로움	첫 구현 비용, 공용 규칙 밖 연출의 처리

구현

공용 시스템이 리치 텍스트 태그를 깨지 않는 타이핑, 슬롯 포커스, 배경만 있는 줄, 뒤로가기 닫기를 처리하고, 이벤트 쪽은 완료 콜백으로 후속 동작을 연결합니다.

결과

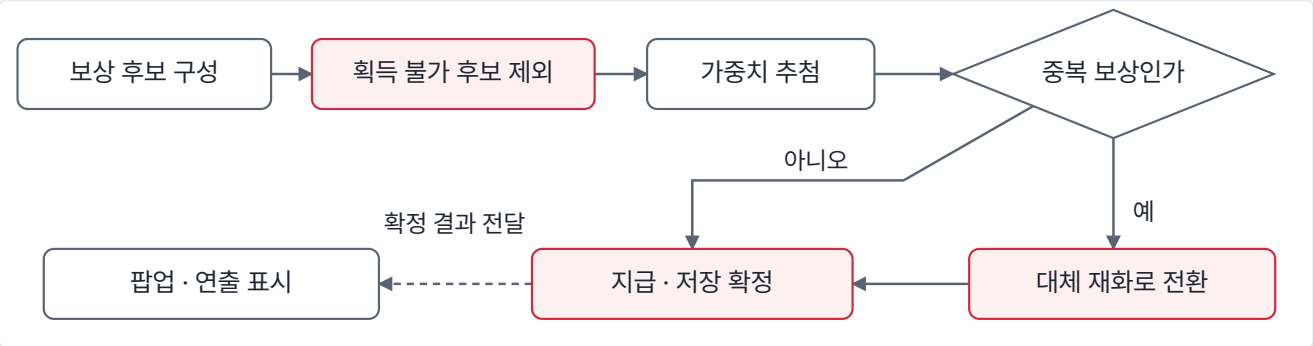
새 이벤트의 대사를 공용 시스템에 올리는 구조로 바꿨습니다. (기록 범위: 제작 시간·QA 비용의 전후 비교 기록은 없음)



공용 대사 시스템 전후(설명용 재구성): 이벤트별 직접 구현 → 데이터·공용 시스템·연출 훅(세부 경로는 자기서술).

보상 지급·저장과 표시를 분리하고, 추첨 전에 획득 불가 후보를 제외

광고 추가 개봉과 가중치 추첨이 섞인 보상 상자에서 연출 변경과 지급·저장 경로를 떼어 놓았습니다.



보상 상자(일반화): 후보 제외·중복 전환·지급 확정 뒤 결과만 표시 계층에 전달.

문제

일일 미션 포인트로 단계별 상자를 열고, 광고로 추가 개봉하며, 스킨·재화를 가중치로 추첨하는 보상입니다. 지급과 표시가 한 흐름에 섞이면 팝업·연출만 바꿔도 지급·저장 경로가 같이 바뀝니다. 또 추첨 후보에 따라 빈 보상·중복 보상이 나올 수 있습니다.

원인

확률표에 이미 다 가진 등급이나 상자 대상이 아닌 스킨이 남으면 빈 보상·중복 보상이 됩니다. 확률·보상 데이터는 라이브 중에도 표 데이터로 바뀝니다.

판단

진행, 보상 결정, 표시를 다른 책임으로 나누고 표시 계층은 확정된 결과만 받게 했습니다. 후보 처리 방식의 비교는 지금 정리한 것입니다.

방식	장점	한계
추첨 뒤 무효면 재추첨	표를 그대로 사용	유효 후보가 0개면 반복이 끝나지 않을 수 있음
추첨 전 후보 제외 (선택)	재추첨 반복을 피하고 유효 후보를 명시	표의 확률이 유효 후보 안의 확률로 바뀜

구현

미션 결과 → 포인트 반영 → 단계 판정 → 유효 후보 구성 → 등급·스킨 추첨 → 중복은 재화로 전환 → 지급·저장 확정 → 팝업·개봉 연출에 결과 전달.

결과

보상 결정·지급·저장을 처리한 결과만 표시 계층으로 넘기고, 후보 제외와 중복 재화 전환을 넣었습니다. (기록 범위: 유효 후보가 모두 빠지는 경우의 반환 정책, 저장 실패 재시도, 표시 확률 검증 기록은 미확인)

함께 맡은 일: 기존 색상 선택을 보존하며 스킨 상태를 여러 화면에 연결

서비스 중인 색상 변경 기능 위에 스킨을 추가했습니다. 스킨 정의는 런타임에 한 번 인덱싱하고, 각 화면은 이 기준을 참조해 상태를 다시 계산합니다.

수정 기록	처리
기본 스킨을 고르면 색상 선택이 풀림	기존 색상 상태를 보존하도록 수정
일부 결과 화면에 스킨 초상화가 빠짐	PvP·레이스 결과 화면에 표시 경로 연결
탭을 옮겨도 설정 창이 닫히지 않음	탭 이동 시 닫힘 처리

회전·스케일이 누적되는 UI에서 월드 위치와 방향 기준으로 OBB 충돌 판정

UGUI 계층 안에서 회전하는 목표물에 핀을 던지는 기간 한정 미니게임의 판정 문제입니다.

문제

회전하는 목표물에 핀을 던져 꽂는 기간 한정 미니게임입니다. 맞은 것처럼 보이는데 실패로 처리되는 경우가 생겼습니다.

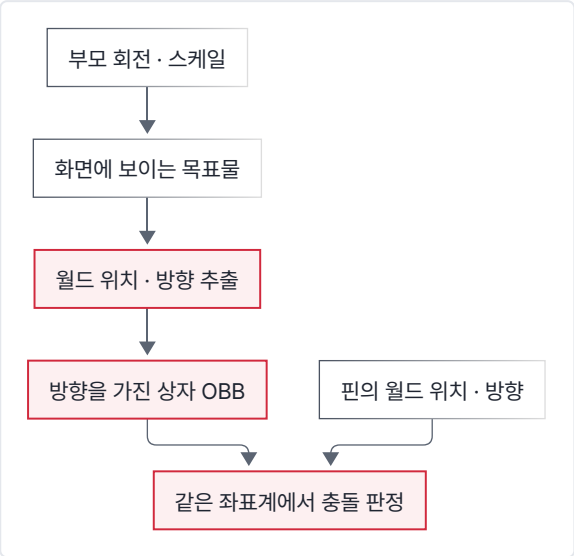
원인

오브젝트가 UGUI 계층에 있어 부모의 회전·스케일이 누적됩니다. UI 사각형 기준 판정은 회전한 목표물의 보이는 모양과 판정 영역이 달랐습니다.

판단

판정에 쓰는 기준 좌표와 방향을 하나로 정했습니다. 판정 방식 비교는 지금 정리한 것입니다.

판정 방식	한계
UI 사각형(축 정렬)	회전하면 보이는 모양과 판정 영역이 어긋남
물리 콜라이더	캔버스 스케일·카메라 설정과 UI 연출을 함께 맞춰야 함
원 근사	길쭉한 핀 모양에 맞지 않음
월드 위치·방향 OBB (선택)	좌표 변환 계산을 직접 관리해야 함



OBB 판정 절차(개념도): 목표물과 핀을 월드 위치·방향 기준의 같은 좌표계에서 판정.

구현

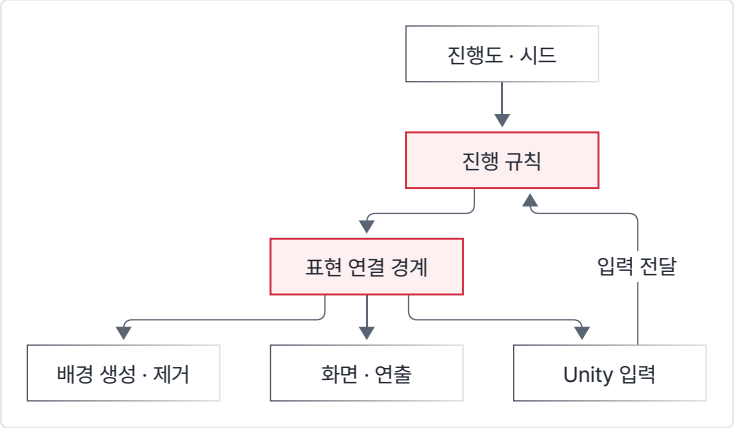
목표물과 핀의 월드 위치·방향을 공통 좌표계에서 비교하도록 판정 기준을 바꿨습니다. 주변 구조는 운영 데이터 파싱·검증, 런타임 모델, 라운드 상태 전환, 판정기로 나눴고, 목표물·등급·회전은 세션 시드와 단계·라운드 값으로 고릅니다.

결과

월드 OBB 판정으로 기준을 바꾸고, 이어하기 광고·스킵·네트워크 예외를 수정했습니다. (기록 범위: 판정 오차의 전후 측정 기록은 없음)

함께 맡은 일: 상승형 미니게임의 진행 규칙과 Unity 표현 분리

높이 구간마다 배경·보상·대화가 바뀌고, 광고 이어하기·부스터·일시정지가 끼어드는 미니게임입니다. 시작·완료·지급·결과 적용은 Unity 의존이 적은 흐름 객체에 두고, 입력·시간·스크롤·연출은 연결 계층으로 나눴습니다. 배경은 진행도 구간과 유저 시드로 결정적으로 고르게 했습니다. 같은 시드라도 데이터 버전이나 난수 소비 순서가 바뀌면 결과가 달라질 수 있습니다.



규칙·표현 경계(일반화): 진행도·시드 규칙과 Unity 입력·배경·연출을 분리.

광고 시청 후 복귀 경로에서 빠지던 보상 저장 수정

문제

메인 화면에서 광고를 보고 티켓을 받는 흐름에서 보상이 저장되지 않는 경로가 있었습니다. 미니게임 이어하기 광고에서도 스킵·네트워크 실패·일시정지 후 종료 같은 예외가 반복됐습니다.

제약

광고 SDK 콜백 시점이 앱의 백그라운드 전환과 엮이고, 라이브 빌드의 수정은 다음 업데이트에 반영됩니다.

판단

광고 보상 경로의 지급 → 저장 → UI 갱신 순서를 점검해 누락된 구간을 찾았습니다.

구현

광고 결과 콜백 → 보상 지급 → 저장 → UI·알림 갱신 순서로 경로를 맞추고, 이어하기 광고의 스킵·실패 예외를 처리했습니다.

결과

광고 보상 경로에서 누락된 저장 처리를 수정했습니다. (기록 범위: 수정 후 재발 추적 기록은 없음)

라이브 운영에서 맡은 그 밖의 일

범위	한 일
이벤트	이벤트 10종 이상 개발·수정, 신규 미니게임을 공통 이벤트 생명주기에 연결
도구·데이터	레벨 에디터 업데이트, 스킨 리소스 폴더 단위 등록 에디터 도구, 현지화 리소스 반영
화면·연출	UI 변경 대응, 튜토리얼·연출 스크립트 수정, 캐릭터 획득 시 프로필 배지 지급 연동
정리·안정화	종료 이벤트 자산 정리, 테스트 기능의 에디터 전용 제한, Crashlytics 리포트 기반 크래시 조사

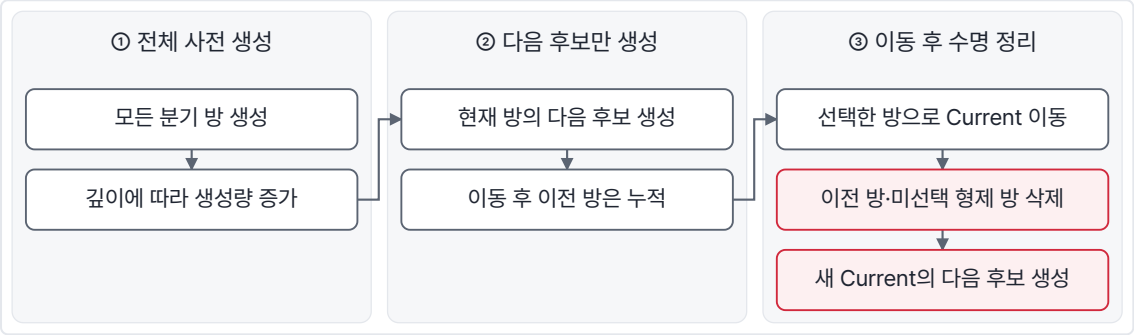
개인·팀 프로젝트 1 · Unity 3D 액션 · 팀 프로젝트

DARK SHIELD

다음 후보 방만 생성하도록 바꾸고, 이동 뒤 이전 방과 선택하지 않은 분기의 오브젝트를 정리했습니다.

기간	2024.11.26~2024.12.12	팀	개발 3인 · 팀장
내 역할	분기 스테이지 생성·정리, 상호작용 표시, 대화 종료 후 후속 동작	기술	Unity 2022.3 · C# · ScriptableObject · NavMeshAgent · Input System
링크	GitHub · 플레이 영상		

1 분기형 포털의 스테이지 생성 범위를 3단계로 변경



스테이지 구조 변경 3단계: 동시 실행이 아니라 커밋 순서대로의 설계 변경.

문제

포털마다 다음 방이 여러 갈래로 갈라집니다. 분기가 늘면 깊이에 따라 미리 만들 스테이지 수가 기하급수적으로 늘 수 있었습니다.

원인

처음 구조는 모든 분기의 방을 미리 만들어 연결했습니다. 씬 전환 없이 한 씬 안에서 플레이어·카메라를 다음 구역으로 옮기는 방식이었습니다.

판단

공개 커밋 순서로 남은 실제 변경 기록입니다.

단계	방식	드러난 한계
①	전체 사전 생성	분기가 깊어질수록 생성량 증가
②	현재 방의 다음 후보만 생성	지나온 방이 계속 쌓임
③ (최종)	이동 뒤 이전 방·미선택 형제 방 삭제	GameObject 정리이며 메모리 상한 보장은 아님

구현

노드에 이전·다음 연결을 두고, 선택한 방으로 현재 노드를 옮긴 뒤 이전 방과 형제 방을 지우고 새 다음 후보를 만듭니다. 관련 코드 StageManager.cs L93–127 · 변경 커밋 db28096 · d475539

결과

다음 후보 방만 만들고, 이동 뒤 이전 방과 미선택 분기의 GameObject를 정리했습니다. (기록 범위: 메모리 사용량 비교 기록은 없음)

2 포털·상점·강화·회복의 입력을 하나의 상호작용 규약으로

주변 탐지 → 공통 인터페이스의 가능 여부 확인 → 실행 순서로 묶고, 월드 좌표를 UI 인디케이터로 바꿔 표시했습니다. 상인과의 대화는 문장 큐가 비면 상점을 여는 후속 동작을 호출합니다. 첫 충돌체를 쓰므로 가장 가까운 대상을 고르지는 않습니다. 관련 코드 Interactor.cs L27–58 · 변경 커밋 df2247b · 5cea1a3



실제 플레이 화면 3컷: 공통 상호작용 규약이 적용된 E 키 상호작용 대상(NPC·포털·오브젝트).

Jumac (주막의 전설)

점수를 키로 쓰던 랭킹을 목록 정렬 방식으로 바꿔 동점 기록도 모두 표시되게 수정했습니다.

항목	내용
기간	2025.01
팀	개발 3명(팀장) + 기획 1명
내 역할	인벤토리 구조·장착 연동, 랭킹 자료구조 수정, 로비 관리 코드 작성 참여
기술	Unity 2022.3 · C# · Firebase Authentication · Realtime Database · Android
링크	GitHub · 플레이 영상

1 랭킹의 동점 기록을 자료구조 선택 문제로 보고 수정

문제

오목·무궁화·제기차기 랭킹에서 같은 점수의 기록을 함께 표시해야 했습니다.

원인

점수를 키로 하는 정렬 사전에 기록을 담아, 같은 점수가 두 번 들어오면 `Add`가 키 중복 예외를 냈습니다.

판단

아래 표는 같은 동점 입력을 두 자료구조가 어떻게 처리하는지 코드 동작으로 정리한 설명용 입력 예입니다.

자료구조	입력 예 A 300 · B 300 · C 200	남는 한계
점수 키 정렬 사전 (이전)	B를 추가할 때 키 중복 예외	점수 하나에 기록 하나
목록 + 점수 내림차순 정렬 (수정)	세 기록 모두 표시	동점도 연속 순번으로 표시

구현

기록을 목록에 모두 담은 뒤 점수 내림차순으로 정렬하고, 게임 종류를 열거형 하나로 묶어 세 미니게임이 같은 랭킹 패널을 쓰게 했습니다. 관련 코드 LeaderBoard.cs L57–76 · 변경 커밋 4ab4463

결과

동점 기록을 모두 목록에 담아 표시하며, 등수는 공동 등수가 아닌 연속 순번입니다.

2 인벤토리·장착 변경을 Presenter가 UI와 DB 저장에 함께 반영



장착·해제 UI와 Firebase 저장값(실제 화면, UID 가림): 장착 상태가 화면과 DB 값에 함께 반영됨.

인벤토리 UI, 장비 슬롯, 스탯 창, 저장값이 따로 갱신되면 장착 상태가 화면과 저장값에서 어긋날 수 있습니다. 실제 장애 기록이 아니라 설계 단계의 예방 목적입니다. Model 이벤트(추가·제거·장착·해제)를 Presenter가 받아 UI 갱신과 DB 저장을 함께 처리하고, 재접속 시 저장값으로 Model을 복원합니다. Presenter가 전역 매니저를 직접 참조해 MVP에 가까운 역할 분리이고, DB 저장 실패 시 롤백·재시도 처리는 확인되지 않습니다. 관련 코드 InventoryPresenter.cs L14–53 · 변경 커밋 1adaf23 · 46c7ecc

개인·팀 프로젝트 3 · VR 멀티 리듬 게임 · 경일게임아카데미 기업 협약 XR 프로젝트

NT

곡 에디터에서 만든 데이터를 노트 생성 경로에 연결하는 변환·로드를 맡았습니다(노트 생성·이동의 DSP 시간 기준은 팀 구조).

항목	내용
기간	2025.02~2025.05
팀	기획·개발 협업 팀 · 개발팀장
내 역할	곡 에디터 데이터 → 노트 데이터 변환 연동, 에디터 데이터 로드·게임 설정
기술	Unity 2020.3 · C# · Meta Quest 2·3 · Photon PUN(팀 멀티)
링크	GitHub · 플레이 영상

에디터에서 만든 곡 데이터를 노트 생성 경로에 연결

문제

곡 에디터에서 만든 곡을 게임에서 플레이하려면, 에디터 데이터를 노트 생성기가 읽는 노트 데이터로 바꾸고 로비에서 곡을 골라 시작하는 경로가 필요했습니다.

제약

노트 생성기는 팀이 `AudioSettings.dspTime`(오디오 엔진 시계) 기준으로 만든 구조로, 노트를 시간순으로 정렬한 뒤 현재 DSP 시각과 비교해 생성합니다.

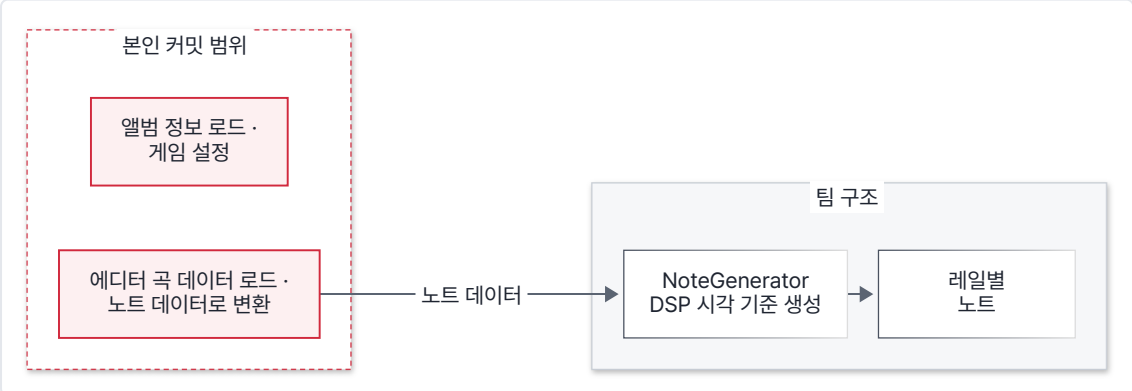
구현

에디터 프로젝트를 로드해 노트 데이터로 바꾸고, 곡·난이도 정보로 게임을 시작하게 연동했습니다. 로비에서는 앨범 정보를 로드해 게임 화면을 설정하고, 고른 앨범의 데이터를 불러와 시작하게 연결했습니다.

결과

에디터에서 만든 곡을 로비에서 골라 플레이하는 경로를 연결했습니다. 로드·게임 설정 커밋(5a91cc0)에는 "에디터에서는 되고 VR 빌드에서는 안 되던 버그" 수정도 기록돼 있습니다(원인은 미기재). (기록 범위: 생성 루프는 팀 코드의 `Task.Delay(1)` 폴링이라 시간 정밀도를 보장하지 않고, 음악·노트 오차 측정 기록은 없음)

구간	구분	근거
에디터 데이터 → 노트 데이터 변환	본인 커밋	3e36a41
에디터 데이터 로드·게임 설정	본인 커밋	5a91cc0
DSP 시각 기준 노트 생성·이동	팀 프로젝트 구조	NoteGenerator.cs L82–136
레일별 타격음 클립 전환	팀 프로젝트 구조	NoteAudioScheduler.cs L17–66



NT 데이터 연결(설명용): 강조한 변환·로드 연결은 본인 커밋, DSP 기준 노트 생성·이동은 팀 구조.

기술·교육·링크

기술 맥락

기술	사용한 곳
C# · Unity	CatJump 업무 전반, DARK SHIELD · Jumac · NT
UGUI · 표 데이터 파싱	CatJump 보상·스킨 화면, 미니게임 운영 데이터
Photon PUN2	CatJump 10인 PvP
Firebase	Jumac 인증·실시간 DB, CatJump 로그·크래시 리포트
C · C++	42서울 과제, IRC 서버(팀), MAZEGAME

교육

기간	과정	내용
2024.09~2025.05	경일게임아카데미	Unity 2D·3D·VR, 기획자 협업 프로젝트
2021.11~2023.11	42서울(이노베이션 아카데미)	C·C++ 과제, 동료 학습

링크

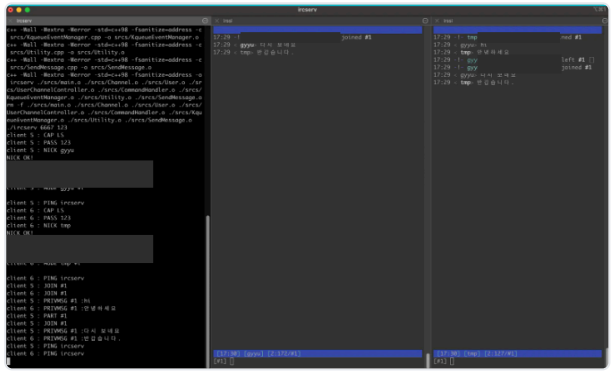
항목	주소
이메일	yugd1211@gmail.com
GitHub	github.com/yugd1211
영상 채널	youtube.com/@k_1q2w3e4r
CatJump	Google Play
DARK SHIELD	GitHub · 영상
Jumac	GitHub · 영상
NT	GitHub · 영상
IRC 서버	GitHub · 영상
MAZEGAME	GitHub · 영상
요식업자로 살아남기	GitHub

부록

IRC 채팅 서버 (팀 프로젝트 구조)

항목	내용
기간	2023.06.06~2023.07.02
팀	42서울 2인 팀
내 역할	확인된 본인 커밋: MODE 오류 메시지 처리 수정(2인 팀, 담당 경계 미표기)
기술	C++98 · BSD Socket · kqueue · 논블로킹 I/O · Docker
링크	GitHub · 시연 영상 · 커밋 2ab63c1

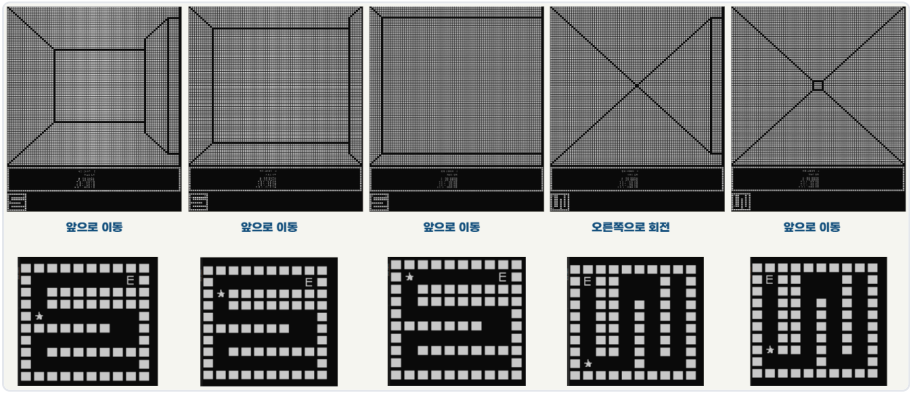
- 논블로킹 소켓과 kqueue 단일 루프로 여러 클라이언트를 처리
- 클라이언트별 수신 버퍼에 누적해 CRLF 단위로 명령 추출
- PASS·NICK·USER 완료 전 상태를 분리하고, 명령은 함수 포인터 맵으로 호출
- 팀은 Docker로 InspIRCd·irssi 기준 환경을 만들고 패킷을 비교해 응답 형식을 맞춤
- `send()` 부분 전송과 재시도는 처리하지 않음



실제 실행 화면(사설 IP 가림): 서버 로그와 irssi 클라이언트 2개의 대화로 서버 동작을 보여 줌.

1인 학습 프로젝트

MAZEGAME · 2024.09.27~2024.10.03 · 1인 · C++ · WinAPI 콘솔 무작위 미로를 만든 직후 BFS로 출구 도달을 검사하고, 실패하면 다시 생성합니다(재시도 상한 없음). 관련 코드 Map.cpp L69–129 · 플레이 영상



MAZEGAME 실제 콘솔 화면: 이동·회전에 따라 다시 그려지는 1인칭 시야 5컷.

요식업자로 살아남기 · 2024.10.30~2024.11.08 · 1인 · Unity 2D · 기획반 제공 기획서 조리대·계산대·테이블마다 반복되던 주변 탐색을 공통 상호작용 영역으로 묶었습니다. 손님이 사라진 뒤 남은 테이블 점유는 두 곳에서 해제하는 방어 코드로 처리했습니다. 관련 코드 InteractiveObject.cs L5–57 · TableManager.cs L24–45